ALA 2 Report

Virtualization using Docker containers: resource allocation and isolation

1. Objective

To install and configure a container-based virtual environment, to understand the fundamentals of virtualization. The activity demonstrates (a) creation, configuration and execution of virtual environments, (b) allocation of CPU, memory and storage to each instance, and (c) isolation between multiple virtual instances.

2. Concepts Used

Virtualization lets one physical computer run several separate environments, each behaving like its own computer. There are two common approaches:

	Virtual machine (VM)	Container (used in this practical)
How it works	A hypervisor runs a complete guest operating system for every VM	Containers share the host operating system kernel and run only the application and its files
Size and speed	Large (GBs) and slower to start	Small (MBs) and starts in seconds
Isolation	Strong, separate kernel per VM	Process-level isolation using Linux namespaces
Resource limits	Set in the hypervisor settings	Enforced by Linux control groups (cgroups)

Docker is a platform for creating and running containers. Each container gets its own file system, hostname, process list and network (isolation through namespaces), and the CPU and memory it may use can be limited (resource control through cgroups). **Docker Compose** describes several containers in one configuration file (docker-compose.yml) and starts them with a single command.

3. Environment

Item	Details
Host computer	Windows 11 laptop, 16 GB RAM
Virtualization	Enabled in the system (confirmed in Task Manager)
Container platform	Docker Desktop 4.98.0, Docker Engine 29.9.2, with the WSL 2 backend (Ubuntu installed in WSL)
Image used	nginx:alpine (small web server image)
Configuration tool	Docker Compose (docker-compose.yml) and a Dockerfile for each instance

4. Design and Configuration

Two virtual instances were created from the same image but with different resources and separate networks.

Setting	Instance 1 (vm-web1)	Instance 2 (vm-web2)
Hostname	web1	web2
CPU limit	0.5 CPU	0.25 CPU
Memory limit	128 MB	64 MB
Fixed-size scratch storage (/scratch)	50 MB (tmpfs)	20 MB (tmpfs)
Persistent storage (/data)	Own volume: web1_data	Own volume: web2_data
Network	net_1 (private)	net_2 (private)
Web port on host	8081	8082

The complete docker-compose.yml and Dockerfile are given in the appendix, and the setup files are submitted as ala2-docker-source.zip.

5. Implementation Steps

1. Enabled WSL 2 with the command `wsl --install` and installed Docker Desktop.
2. Verified the installation with `docker --version` and `docker run hello-world` (Figure 1).
3. Prepared the project folder containing docker-compose.yml and one folder per instance (web1, web2), each with a Dockerfile and an index.html page.
4. Created and started both instances with `docker compose up -d --build` and checked them with `docker ps` (Figures 2 to 4).
5. Checked the memory limits using `docker stats --no-stream` (Figure 5).
6. Generated CPU load inside vm-web1 and observed that the CPU use stays near the 0.5 CPU limit (Figure 6).
7. Checked storage allocation with `df -h /scratch` and tested isolation using hostname, file and ping tests (Figure 7).
8. Stopped and removed the instances with `docker compose down` (Figure 8).

6. Results and Screenshots

6.1 Docker installation

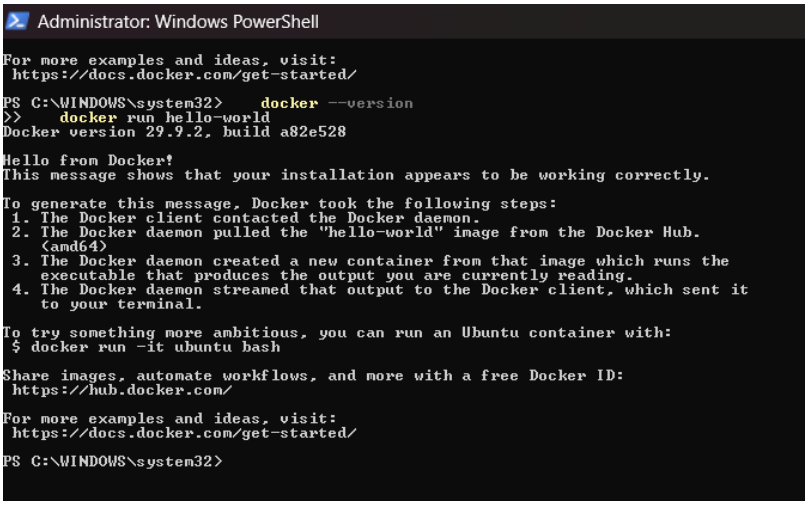


Figure 1: Docker version and the hello-world test container ran successfully

The message "Hello from Docker!" confirms that the Docker client, the Docker daemon and the container runtime are working correctly.

6.2 Creation and execution of the virtual instances

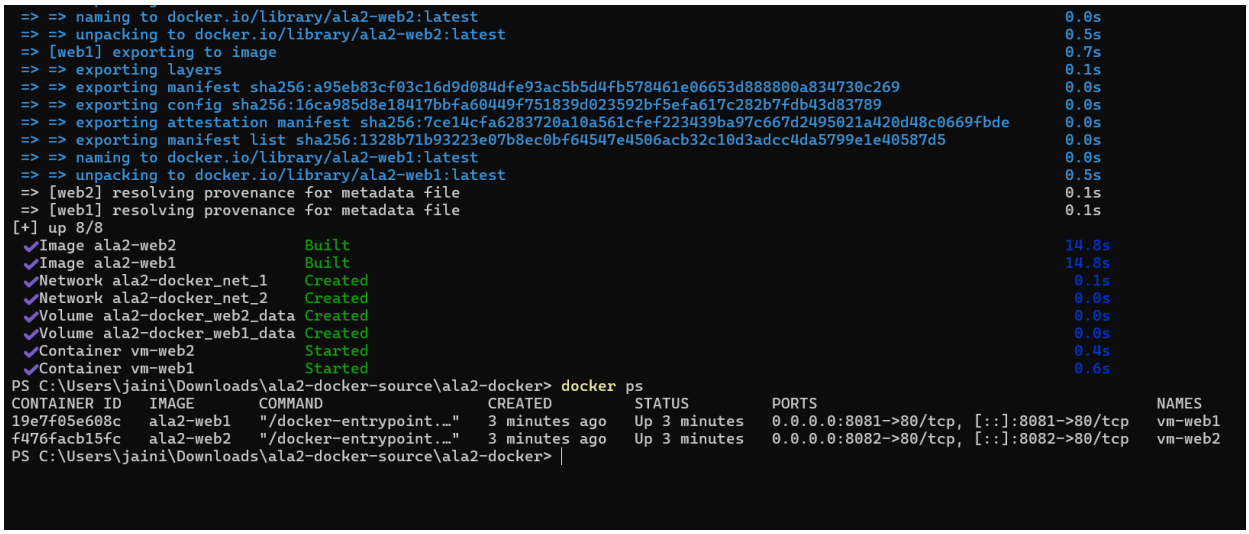


Figure 2: docker compose up creates images, networks, volumes and containers; docker ps shows both running

Compose built the images ala2-web1 and ala2-web2, created the networks ala2-docker_net_1 and ala2-docker_net_2, created one volume for each instance, and started the containers vm-web1 and vm-web2. `docker ps` shows both with the status **Up**, published on ports 8081 and 8082.

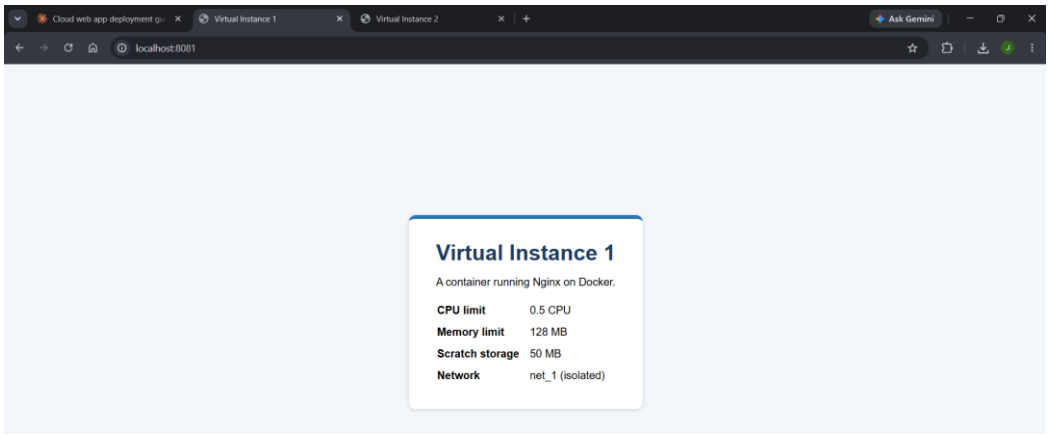


Figure 3: Virtual Instance 1 running at localhost:8081

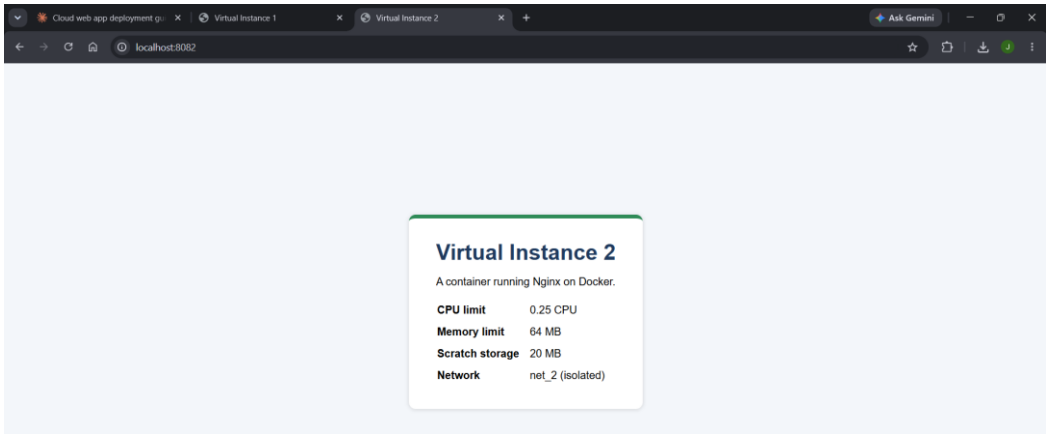


Figure 4: Virtual Instance 2 running at localhost:8082

Both instances serve their own web page at the same time, each showing its own configured limits. This shows that several virtual instances can run together on one host.

6.3 Resource allocation: memory

```
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker>    docker stats --no-stream
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O        PIDS
19e7f05e608c   vm-web1   0.00%     14.48MiB / 128MiB   11.31%    1.7kB / 126B    9.66MB / 4.1kB   17
f476facb15fc   vm-web2   0.00%     14.68MiB / 64MiB    22.94%    1.7kB / 126B    4.1kB / 4.1kB    17
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker>    docker exec -d vm-web1 sh -c "while true; do :; done"
```

Figure 5: docker stats showing the memory limit of each instance

Instance	Memory used	Memory limit	Percent used
vm-web1	14.48 MiB	128 MiB	11.31%
vm-web2	14.68 MiB	64 MiB	22.94%

Each instance has its own memory limit (128 MiB and 64 MiB) even though both run the same software. If an instance tries to use more than its limit, Docker restricts it and does not let it take memory from the other instance or the host.

6.4 Resource allocation: CPU

```
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker>    docker exec -d vm-web1 sh -c "while true; do :; done"
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker>    docker stats --no-stream
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O        PIDS
19e7f05e608c   vm-web1   49.78%    15.81MiB / 128MiB   12.35%    1.7kB / 126B    11.2MB / 4.1kB   18
f476facb15fc   vm-web2   0.00%     14.68MiB / 64MiB    22.94%    1.7kB / 126B    4.1kB / 4.1kB    17
```

Figure 6: CPU load generated in vm-web1 stays near its 0.5 CPU limit

A loop was started inside vm-web1 that tries to use as much CPU as possible. Without a limit it would use a full core (100%). `docker stats` shows vm-web1 at **49.78%**, which matches the configured limit of 0.5 CPU, while vm-web2 stays at 0.00%. The CPU load was removed afterwards by restarting vm-web1.

6.5 Resource allocation: storage, and isolation

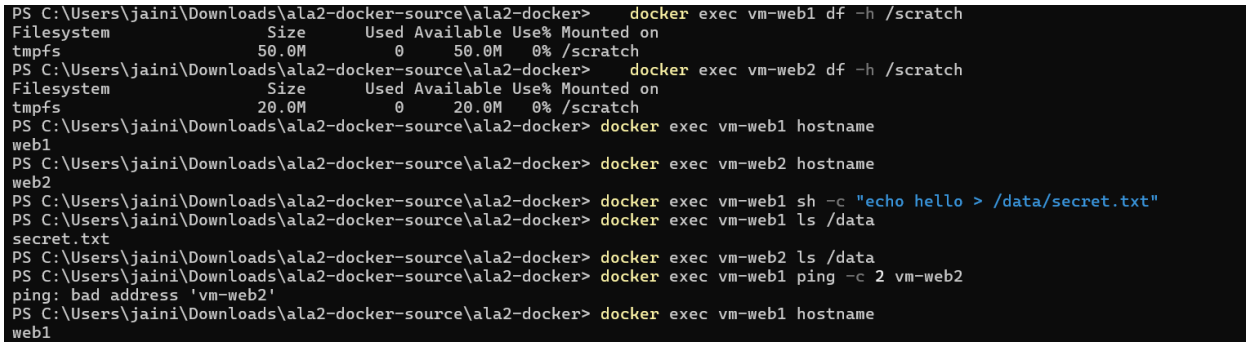


Figure 7: Storage sizes of both instances and the isolation tests

Test	Command	Result
Storage allocation	df -h /scratch	vm-web1: 50.0M, vm-web2: 20.0M (separate tmpfs areas)
Separate hostname	hostname	vm-web1 returned web1, vm-web2 returned web2
Separate file system	write secret.txt in vm-web1, then ls /data in both	secret.txt is present only in vm-web1; /data of vm-web2 is empty
Separate network	ping from vm-web1 to vm-web2	Failed (bad address): they are on different private networks

A file created in one instance cannot be seen from the other, the two have different identities, and they cannot reach each other over the network. This demonstrates isolation between the virtual instances.

6.6 Clean-up

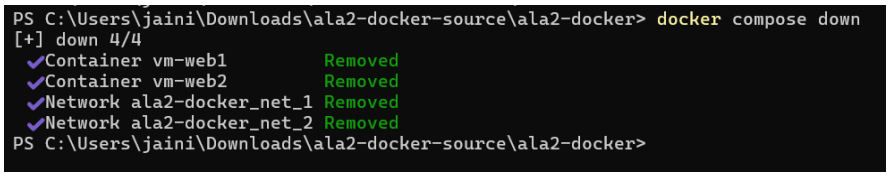


Figure 8: docker compose down removes the containers and networks

The containers and networks were removed. The named volumes web1_data and web2_data are not removed by this command, so stored data is kept for the next start.

7. Observations

- Both instances were created from one image yet received different CPU, memory and storage limits.
- The CPU limit was enforced: the load stayed at about 50% for an instance limited to 0.5 CPU.
- Containers started in seconds and used very little memory (about 15 MiB each) compared with a full virtual machine.
- Isolation worked for the file system, hostname and network.

- The 50 MB and 20 MB scratch areas are tmpfs (memory-backed) and are cleared when the container stops. The /data volumes are persistent but have no size cap in this setup.

8. Challenges Faced

Challenge	Solution
docker run failed with an error that the Docker engine pipe could not be found	Docker Desktop was not running. After starting Docker Desktop and waiting for the message "Engine running", the commands worked
Ubuntu in WSL asked for a username and password during installation	Created a username and password; typing the password shows no characters, which is normal in Linux

9. Conclusion

In this practical I installed Docker, created two container-based virtual instances with Docker Compose, and gave each a different CPU, memory and storage allocation. I verified the limits with docker stats and df, and showed isolation through separate hostnames, file systems and networks. The activity helped me understand virtualization, the difference between virtual machines and containers, and how resources can be shared and controlled on one host.

Appendix A: Configuration files

docker-compose.yml

```
# ALA 2 - Virtualization using Docker containers
# Two isolated virtual instances with different CPU, memory and storage limits.

services:
  web1:
    build: ./web1
    image: ala2-web1
    container_name: vm-web1
    hostname: web1
    cpus: 0.5            # CPU limit: half of one CPU core
    mem_limit: 128m      # Memory limit
    ports:
      - "8081:80"
    volumes:
      - web1_data:/data  # persistent storage, private to this instance
      - type: tmpfs      # fixed-size scratch storage (50 MB)
        target: /scratch
        tmpfs:
          size: 52428800
    networks:
      - net_1

  web2:
    build: ./web2
    image: ala2-web2
    container_name: vm-web2
    hostname: web2
    cpus: 0.25           # CPU limit: quarter of one CPU core
    mem_limit: 64m       # Memory limit
    ports:
      - "8082:80"
    volumes:
      - web2_data:/data
      - type: tmpfs      # fixed-size scratch storage (20 MB)
        target: /scratch
        tmpfs:
          size: 20971520
    networks:
      - net_2
```

```
volumes:
  web1_data:
  web2_data:

networks:
  net_1:
  net_2:
```

web1/Dockerfile (web2/Dockerfile is identical)

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
```

Appendix B: Complete command session

```
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O         PIDS
19e7f05e608c   vm-web1   0.00%     14.48MiB / 128MiB   11.31%    1.7kB / 126B    9.66MB / 4.1kB    17
f476facb15fc   vm-web2   0.00%     14.68MiB / 64MiB    22.94%    1.7kB / 126B    4.1kB / 4.1kB     17
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker>    docker stats --no-stream
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O         PIDS
19e7f05e608c   vm-web1   0.00%     14.48MiB / 128MiB   11.31%    1.7kB / 126B    9.66MB / 4.1kB    17
f476facb15fc   vm-web2   0.00%     14.68MiB / 64MiB    22.94%    1.7kB / 126B    4.1kB / 4.1kB     17
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker>    docker exec -d vm-web1 sh -c "while true; do :; done"
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker>    docker stats --no-stream
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O         PIDS
19e7f05e608c   vm-web1   49.78%    15.81MiB / 128MiB   12.35%    1.7kB / 126B    11.2MB / 4.1kB    18
f476facb15fc   vm-web2   0.00%     14.68MiB / 64MiB    22.94%    1.7kB / 126B    4.1kB / 4.1kB     17
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker>    docker compose restart web1
[+] restart 0/1
 - Container vm-web1 Restarting                                                                                  0.8s
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker>    docker exec vm-web1 df -h /scratch
Filesystem                Size      Used Available Use% Mounted on
tmpfs                    50.0M         0     50.0M   0% /scratch
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker>    docker exec vm-web2 df -h /scratch
Filesystem                Size      Used Available Use% Mounted on
tmpfs                    20.0M         0     20.0M   0% /scratch
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker> docker exec vm-web1 hostname
web1
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker> docker exec vm-web2 hostname
web2
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker> docker exec vm-web1 sh -c "echo hello > /data/secret.txt"
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker> docker exec vm-web1 ls /data
secret.txt
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker> docker exec vm-web2 ls /data
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker> docker exec vm-web1 ping -c 2 vm-web2
ping: bad address 'vm-web2'
PS C:\Users\jaini\Downloads\ala2-docker-source\ala2-docker> docker exec vm-web1 hostname
web1
```

Figure A1: Complete terminal session (stats, CPU test, restart, storage and isolation tests)